


**UNIVERSITATEA TEHNICĂ A MOLDOVEI**

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

## LUCRARE DE LABORATOR NR. 5

# The File System with Your Own Hands

Disciplina: Sisteme de Operare

**A efectuat:**
student gr. FAF-243
Daniel Nicorici

**Au verificat:**
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

# Cuprins

**Purpose of the work** **2**

**1 Part 1. The tree and the file types** **2**

**2 Part 2. Inodes and links** **3**

**3 Part 3. Permissions** **5**

**4 Part 4. Disks and the kernel as files** **6**

**5 Stretch (optional)** **8**

**Conclusions** **9**

## Purpose of the work

The purpose of this lab is to check, on my own Linux laptop, everything from the lecture about file systems: the directory tree and the file types, the inodes that stand behind file names, hard and symbolic links, permissions, and the disks underneath, including the parts of the kernel that are shown as files under `/proc`. There is no programming: only commands, careful observation and short explanations.

The work was done on Arch Linux (kernel 7.0.12, ext4 root file system) in GNU bash 5.3.15, as my normal user and not as root, because root ignores most permission checks and the experiments in Part 3 would not show anything. All experiments ran in a fresh folder `~/os-lab5-run`.

```
$ whoami
Nico
$ mkdir -p ~/os-lab5-run && cd ~/os-lab5-run
```

# 1 Part 1. The tree and the file types

```
$ ls /
bin   dev  home  lib64       luminis-front  opt   root  sbin  sys  usr
boot  etc  lib   lost+found  mnt            proc  run   srv   tmp  var
```

Figure 1.1 – The root of the tree

```
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx  1 root root    7 Oct 12  2025 /bin -> usr/bin
crw-rw-rw-  1 root root 1, 3 Oct 12 13:13 /dev/null
drwxr-xr-x 92 root root 4096 Oct 12 15:39 /etc
-rw-r--r--  1 root root 1647 Sep 17 14:29 /etc/passwd
drwxrwxrwt 29 root root  920 Oct 12 16:09 /tmp
```

Figure 1.2 – The type of five paths

```
$ ls -l /dev | head
total 0
drwxr-xr-x   2 root root           60 Oct 12 13:13 accel
crw-r--r--   1 root root    10,   235 Oct 12 13:13 autofs
drwxr-xr-x   2 root root          160 Oct 12 13:13 block
crw-rw----   1 root disk    10,   234 Oct 12 13:13 btrfs-control
drwxr-xr-x   3 root root           60 Oct 12 13:13 bus
drwxr-xr-x   2 root root         5640 Oct 12 15:27 char
crw-------   1 root root     5,     1 Oct 12 13:13 console
lrwxrwxrwx   1 root root           11 Oct 12 13:13 core -> /proc/kcore
drwxr-xr-x  18 root root          360 Oct 12 13:13 cpu
$ ls -l /dev | grep '^b'
brw-rw----   1 root disk   259,     0 Oct 12 13:13 nvme0n1
brw-rw----   1 root disk   259,     1 Oct 12 13:13 nvme0n1p1
brw-rw----   1 root disk   259,     2 Oct 12 13:13 nvme0n1p2
brw-rw----   1 root disk   259,     3 Oct 12 13:13 nvme0n1p3
brw-rw----   1 root disk   259,     4 Oct 12 13:13 nvme0n1p4
brw-rw----   1 root disk   259,     5 Oct 12 13:13 nvme0n1p5
```

Figure 1.3 – Devices in /dev

```
$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
    ↪ interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=
    ↪ c988ae960e91ea3f9f7b9cbbc2e3e4ffc0353796, for GNU/Linux 4.4.0, stripped
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
```

Figure 1.4 – file looks at the content, not the name

```
$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
1.4G    /var/log
20G     /usr
```

Figure 1.5 – Disk usage of /var/log and /usr

**Observations:**

1. **Types from the first letter of `ls -l`.** `/etc` is `d`, a directory; `/etc/passwd` is `-`, a regular file; `/bin` is `l`, a symbolic link; `/dev/null` is `c`, a character device; `/tmp` is `d`, a directory (its permissions end in `t`, explained in Part 3).
2. **What `/bin` is.** It is a symbolic link to `usr/bin`. Arch keeps all programs in `/usr/bin`, and the old path `/bin` survives only as a link so that old scripts keep working; `file` confirms it: `symbolic link to usr/bin`.
3. **Devices.** A character device: `/dev/null` (`c`, numbers `1, 3`), a byte stream that discards everything written to it. A block device: `/dev/nvme0n1` (`b`, numbers `259, 0`), my whole SSD, read and written in blocks; `nvme0n1p1` to `nvme0n1p5` are its partitions. The two numbers shown instead of a size are the major number (which driver) and the minor number (which device).
4. **`/var/log` vs. `/usr`.** `/usr` uses 20 GB and `/var/log` 1.4 GB, so `/usr`, which holds every installed program and library, is about 14 times bigger.

# 2 Part 2. Inodes and links

```
$ echo hello > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 259,5   Inode: 4194703     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/    Nico)   Gid: ( 1000/    Nico)
Access: 2026-10-12 16:10:08.938495542 +0300
Modify: 2026-10-12 16:10:08.938495542 +0300
Change: 2026-10-12 16:10:08.938495542 +0300
 Birth: 2026-10-12 16:10:08.938495542 +0300
$ ls -li notes.txt
4194703 -rw-r--r-- 1 Nico Nico 6 Oct 12 16:10 notes.txt
```

Figure 2.1 – An inode, and a name that is not part of it

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
4194703 -rw-r--r-- 2 Nico Nico 6 Oct 12 16:10 hard.txt
```

```
4194703 -rw-r--r-- 2 Nico Nico 6 Oct 12 16:10 notes.txt
$ rm notes.txt
$ cat hard.txt ; ls -li hard.txt
hello
4194703 -rw-r--r-- 1 Nico Nico 6 Oct 12 16:10 hard.txt
```

Figure 2.2 – Hard link, then removing the original

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
4194703 -rw-r--r-- 1 Nico Nico 6 Oct 12 16:10 hard.txt
4194705 lrwxrwxrwx 1 Nico Nico 8 Oct 12 16:10 soft.txt -> hard.txt
$ cat soft.txt
hello
$ rm hard.txt
$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 Nico Nico 8 Oct 12 16:10 soft.txt -> hard.txt
```

Figure 2.3 – Symbolic link, then removing its target

```
$ mkdir d ; ls -ld d
drwxr-xr-x 2 Nico Nico 4096 Oct 12 16:10 d
$ mkdir d/x d/y ; ls -ld d
drwxr-xr-x 4 Nico Nico 4096 Oct 12 16:10 d
$ ls -ai d d/x
d:
4194703 .  4194700 ..  4194708 x  4194711 y

d/x:
4194708 .  4194703 ..
```

Figure 2.4 – Link counts of a directory

```
$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-r--r-- 1 Nico Nico 2 Oct 12 16:10 one.txt
4.0K    one.txt
$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-r--r-- 1 Nico Nico 1.0G Oct 12 16:10 big.img
0       big.img
$ rm big.img
```

Figure 2.5 – Size is not space

**Observations:**

1. **Inode and link count in `stat`.** `Inode: 4194703` is the inode number, the identity of the file on the disk. `Links: 1` tells how many names (directory entries) point to that inode. `ls -li` shows the same two numbers: the inode in the first column and the link count right after the permissions.

2. **Why `hard.txt` survived `rm notes.txt`.** `ln` did not copy anything: it added a second directory entry pointing to the same inode, so both names show inode 4194703 and the link count became 2. `rm` removes only a *name*: it deleted the entry `notes.txt` and lowered the count to 1, while the inode and its data stayed, still reachable through `hard.txt`. The data is freed

only when the link count reaches 0 and no program has the file open.

3. **Why `cat soft.txt` failed.** A symbolic link is a separate file with its own inode (4194705) whose only content is the path text `hard.txt`. After `rm hard.txt` no file with that name exists, so following the link leads nowhere (a dangling link), although `soft.txt` itself still exists. Its size is exactly 8 bytes because `hard.txt` has 8 characters. The inode number 4194703 was freed when its last name was removed, and the very next `mkdir d` received it again, as Figure 2.4 shows.

4. **Directory links.** A new directory has 2 links: its name `d` in the parent directory, and its own entry `.`, which points to itself. Each subdirectory adds one more, because its `..` entry points back to `d`: with `x` and `y` the count becomes 4. `ls -ai` proves it: `d/.` and `d/x/..` both have inode 4194703.

5. **Size vs. space.** `one.txt` contains 2 bytes (`x` and a newline), but the file system hands out space in whole 4 KiB blocks (`IO Block: 4096`), so even 2 bytes occupy one block: 4.0K. `truncate -s 1G` only set the length of `big.img` to 1 GB without writing anything, which creates a sparse file: the missing parts are holes that read as zeros but have no blocks behind them, so `du` reports 0.

# 3 Part 3. Permissions

```
$ echo secret > s.txt
$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 Nico Nico 7 Oct 12 16:10 s.txt
$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt ; cat s.txt
secret
```

Figure 3.1 – Permissions of a file

```
$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0022
-rw-r--r-- 1 Nico Nico    0 Oct 12 16:10 new.txt
drwxr-xr-x 2 Nico Nico 4096 Oct 12 16:10 newdir
```

Figure 3.2 – umask and the permissions of new files

```
$ mkdir box ; echo hi > box/f
$ chmod u=r,go= box  ; ls box ; cat box/f
ls: cannot access 'box/f': Permission denied
f
cat: box/f: Permission denied
$ \ls box
f
$ chmod u=x,go= box  ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
$ chmod 755 box
```

Figure 3.3 – The box experiment: read-only and execute-only directory

```
$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 84864 Jun 13 00:35 /usr/bin/passwd
drwxrwxrwt 29 root root 920 Oct 12 16:09 /tmp
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw------- 1 root root 846 Sep 17 14:29 /etc/shadow
```

Figure 3.4 – Special bits and a file of another user

**Observations:**

1. **600, 644, 755 and 700.** Each digit is a sum of r = 4, w = 2, x = 1, for the owner, the group and the others.
   - `600` = `rw-------`: private files only I may read and change, such as my SSH key `~/.ssh/id_ed25519`.
   - `644` = `rw-r--r--`: ordinary files everyone may read but only I may change, such as `/etc/passwd` or my notes.
   - `755` = `rwxr-xr-x`: programs and directories everyone may use, such as `/usr/bin/ls` or `newdir`.
   - `700` = `rwx------`: a private directory or script, such as `~/.ssh`.
2. **Why owning `s.txt` did not help.** The kernel decides by the permission bits: because I am the owner, the *owner* bits apply to me, and after `chmod 000` they are `---`. Ownership only gives the right to change the permissions, which is how `chmod u+r` made the file readable again. Only root ignores the bits.
3. **umask.** My umask is `0022`: the bits it lists are removed from every new file. New files start from `666`, so `new.txt` gets `644` = `rw-r--r--`; new directories start from `777`, so `newdir` gets `755` = `rwxr-xr-x`. In both cases the write permission of the group and of the others was taken away.
4. **The box experiment.** A directory is a list of names. `r` allows reading that list: with read-only `box`, `ls` still printed the name `f`, but `cat box/f` was denied. The extra error from `ls` comes from my alias `ls --color=auto`, which looks inside every entry to colour it; the plain `\ls box` printed `f` without any error. `x` allows passing through the directory to reach a file whose name is already known: with execute-only `box`, `ls` could not open the directory, but `cat box/f` printed `hi`.
5. **`s` and `t`.** The `s` in `-rwsr-xr-x` of `/usr/bin/passwd` is the setuid bit: the program runs with the rights of its owner, root, whoever starts it. `passwd` needs it because passwords are stored in `/etc/shadow`, which is `-rw------- root`, so a normal user cannot even read it, as the failed `cat` shows; `passwd` itself makes sure that a user changes only their own password. The `t` in `drwxrwxrwt` of `/tmp` is the sticky bit: everyone may create files there, but only the owner of a file may delete or rename it.

# 4 Part 4. Disks and the kernel as files

```
$ lsblk -i
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0 953.9G  0 disk
|-nvme0n1p1 259:1    0   100M  0 part /boot/efi
```

```
|-nvme0n1p2 259:2    0    16M  0 part
|-nvme0n1p3 259:3    0 247.6G  0 part
|-nvme0n1p4 259:4    0 510.8G  0 part
`-nvme0n1p5 259:5    0 195.3G  0 part /
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p5  192G  162G   20G  89% /
devtmpfs         16G     0   16G   0% /dev
tmpfs            16G  6.6M   16G   1% /dev/shm
efivarfs        128K   57K   67K  47% /sys/firmware/efi/efivars
tmpfs           6.2G  7.3M  6.2G   1% /run
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs            16G  4.2M   16G   1% /tmp
/dev/nvme0n1p1   96M   86M   11M  90% /boot/efi
tmpfs           3.1G  5.0M  3.1G   1% /run/user/1000
$ mount | grep " / "
/dev/nvme0n1p5 on / type ext4 (rw,relatime)
```

Figure 4.1 – Disks, partitions and the root file system

```
$ grep -c processor /proc/cpuinfo
16
$ lscpu | grep -E "^(Core|Thread|Socket)"
Thread(s) per core:                      2
Core(s) per socket:                      8
Socket(s):                               1
$ head -3 /proc/meminfo
MemTotal:       32124248 kB
MemFree:        14917128 kB
MemAvailable:   23347136 kB
$ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   448626
Ngid:   0
$ ls -l /proc/self/fd
total 0
lrwx------ 1 Nico Nico 64 Oct 12 16:10 0 -> /dev/pts/7
lrwx------ 1 Nico Nico 64 Oct 12 16:10 1 -> /dev/pts/7
lrwx------ 1 Nico Nico 64 Oct 12 16:10 2 -> /dev/pts/7
lr-x------ 1 Nico Nico 64 Oct 12 16:10 3 -> /proc/448637/fd
```

Figure 4.2 – The kernel shown as files in /proc

```
nvme0n1   953.9 GB NVMe SSD (block device 259:0)
|-- nvme0n1p1    100 MB  -> /boot/efi   EFI boot partition
|-- nvme0n1p2     16 MB     not mounted
|-- nvme0n1p3  247.6 GB     not mounted
|-- nvme0n1p4  510.8 GB     not mounted
`-- nvme0n1p5  195.3 GB  -> /           ext4, 162 GB of 192 GB used (89%)
```

Figure 4.3 – Drawing of my disk

**Observations:**

1. **My disk.** The only disk is `nvme0n1`, a 953.9 GB NVMe SSD, with five partitions (Figure 4.3). Partition 5, `nvme0n1p5` (195.3 GB), is mounted at `/` with the ext4 file system, and partition 1 (100 MB) is the EFI boot partition at `/boot/efi`. Partitions 2 to 4 are not used by Linux; they belong to another operating system on the same disk.
2. **How full the root file system is.** 162 GB of 192 GB are used, so it is 89% full, with 20 GB available.
3. **CPU and RAM according to `/proc`.** `/proc/cpuinfo` lists 16 processors. These are logical CPUs: `lscpu` shows 8 physical cores with 2 threads each. `/proc/meminfo` reports `MemTotal: 32124248 kB`, about 32 GB (30.6 GiB) of RAM.
4. **The program in `/proc/self/status`.** It showed `Name: head`, in state `R (running)`. `/proc/self` always refers to the process that opens it, and the process that opened `/proc/self/status` was `head` itself. In the same way, `ls -l /proc/self/fd` showed the open files of `ls`: descriptors 0, 1 and 2 point to my terminal `/dev/pts/7`, and descriptor 3 is the directory `/proc/448637/fd` that `ls` had open while listing it.

# 5 Stretch (optional)

```
$ find / -xdev -perm -4000 -type f 2>/dev/null
/usr/lib/electron12/chrome-sandbox
/usr/lib/electron40/chrome-sandbox
/usr/lib/Xorg.wrap
/usr/lib/claude-desktop/chrome-sandbox
/usr/lib/dbus-daemon-launch-helper
/usr/lib/libgtop/libgtop_server2
/usr/lib/electron39/chrome-sandbox
/usr/lib/qemu/qemu-bridge-helper
/usr/lib/ssh/ssh-keysign
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/su
/usr/bin/pkexec
/usr/bin/chage
/usr/bin/sudo
/usr/bin/gpasswd
/usr/bin/suexec
/usr/bin/passwd
/usr/bin/ksu
/usr/bin/fusermount3
/usr/bin/expiry
/usr/bin/mount
/usr/bin/sg
/usr/bin/newgrp
/usr/bin/umount
/usr/bin/unix_chkpwd
/usr/bin/nvidia-modprobe
/usr/bin/fusermount
/opt/github-desktop/chrome-sandbox
/opt/Discord/chrome-sandbox
```

Figure 5.1 – (b) The setuid programs on my laptop

```
$ head -c 100M /dev/zero > bigfile
```

```
$ df -m .
Filesystem     1M-blocks   Used Available Use% Mounted on
/dev/nvme0n1p5    195777 165404     20357  90% /
$ exec 3< bigfile
$ rm bigfile
$ ls -l /proc/$$/fd
total 0
lrwx------ 1 Nico Nico 64 Oct 12 16:09 0 -> /dev/pts/7
lrwx------ 1 Nico Nico 64 Oct 12 16:10 1 -> /dev/pts/7
lrwx------ 1 Nico Nico 64 Oct 12 16:10 2 -> /dev/pts/7
lrwx------ 1 Nico Nico 64 Oct 12 16:10 255 -> /dev/pts/7
lr-x------ 1 Nico Nico 64 Oct 12 16:10 3 -> '/home/Nico/os-lab5-run/bigfile (deleted)'
$ df -m .
Filesystem     1M-blocks   Used Available Use% Mounted on
/dev/nvme0n1p5    195777 165404     20357  90% /
$ exec 3<&-
$ df -m .
Filesystem     1M-blocks   Used Available Use% Mounted on
/dev/nvme0n1p5    195777 165304     20457  89% /
```

Figure 5.2 – (c) Deleting a file that is still open

**Observations:**

(b) **Setuid programs.** The search found 30 setuid programs. The classic ones are there: `passwd`, `su`, `sudo`, `chsh`, `newgrp`, `mount` and `umount`, which all need root rights for one specific job. My laptop also has several that are not typical for a lecture slide and come from the applications I installed: the `chrome-sandbox` helpers of Electron apps (Discord, GitHub Desktop, Claude), the `nvidia-modprobe` helper of the NVIDIA driver and `qemu-bridge-helper` from QEMU.

(c) **Deleting an open file.** `exec 3< bigfile` made the shell itself open the 100 MB file as descriptor 3, and `$$` is the shell's own process ID. After `rm bigfile` the link in `/proc/$$/fd` still points to the file, marked `(deleted)`: the name is gone, but the inode is still in use. The space was *not* freed by `rm` (165 404 MB used before and after); it was freed only when `exec 3<&-` closed the descriptor, after which the used space dropped by exactly 100 MB to 165 304 MB.

# Conclusions

This lab showed that a file on Linux is really two separate things: an inode, which holds the data, the size, the owner and the permissions, and one or more names, which are only entries in directories pointing to that inode. Everything I observed follows from this split. A hard link is just another name for the same inode, so deleting the "original" loses nothing, while a symbolic link is a separate small file that stores a path and breaks when that path disappears. Even directory link counts are names: the parent's entry, the directory's own `.` and the `..` of every subdirectory. Data is freed only when no name and no open file refers to the inode, which the deleted but still open file demonstrated directly.

The permission experiments showed that the kernel checks the permission bits rather than mere ownership, that `r` and `x` mean different things on a directory, and that the umask decides the default permissions of new files. Special bits such as setuid and the sticky bit solve real problems like changing a password or sharing `/tmp`. Finally, `lsblk`, `df` and `/proc` showed that disks, partitions, the CPU, the memory and even the current process are all presented to me as files, which is the core idea of Unix.